

Improving and Optimizing NeCT for NeCTv2

Master's Thesis

Department of Physics / Department of Computer Science
Norwegian University of Science and Technology (NTNU)

Author Name

Supervisor: *Supervisor Name*

April 4, 2026

Abstract

[TODO: Write abstract last. Summarise motivation, methods, key results, and conclusions in roughly 250 words.]

Acknowledgements

[TODO: Acknowledgements.]

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Objectives	1
1.3	Thesis Structure	1
2	Background	2
2.1	Neural Networks	2
2.1.1	Convolutional Neural Networks	4
2.1.2	Implicit Neural Representations	5
2.1.3	Neural Radiance Fields (NeRF)	7
2.2	X-ray Computed Tomography	11
2.2.1	Physical Principles	11
2.2.2	Classical Reconstruction Algorithms	12
2.2.3	4D-CT and Dynamic Imaging	14
2.3	NeCT: Neural Computed Tomography	16
2.3.1	NeCT Pipeline Overview	16
2.3.2	Static 3D Reconstruction: Improved NAF	17
2.3.3	Dynamic 4D Reconstruction: NeCT QuadCubes	19
2.3.4	Hybrid Golden-Section Sampling	20
3	Encoder Size and Performance	22
3.1	Experimental Setup	22
3.2	Effect of Total Parameter Count	22
3.3	Effect on Reconstruction Time	22
3.4	Precision and Accuracy Metrics	22
3.5	Non-Uniform Encoder Sizing	23
4	Encoder Architecture	24
4.1	Motivation and Design Space	24
4.2	QuadCubes (Baseline)	24
4.3	TriCubes	24

4.4	SexCubes (Six Encoders)	24
4.5	Single Encoder	25
4.6	Combined Cubes Method	25
4.7	Comparative Analysis	25
5	Continuous Scanning	26
5.1	Motivation	26
5.2	Forward Model for Continuous Scanning	26
5.3	Static Dataset Experiments	27
5.4	Dynamic Dataset Experiments	27
5.5	Discussion	27
6	Conclusion	28
6.1	Summary of Contributions	28
6.2	Limitations and Future Work	28
A	Supplementary Experimental Details	33
B	NeCTv2 Hyperparameter Reference	34

List of Figures

List of Tables

2.1	Structural analogy between NeRF and attenuation-contrast CT. . . .	8
-----	--	---

Chapter 1

Introduction

[TODO: Motivate the problem: limitations of conventional 4D-CT, the promise of implicit neural representations, and the gap that NeCTv2 addresses.]

1.1 Background and Motivation

[TODO:]

1.2 Research Objectives

[TODO: List the concrete research questions / contributions of the thesis.]

1.3 Thesis Structure

[TODO: One paragraph guiding the reader through the chapters.]

Chapter 2

Background

[TODO: This chapter builds the theoretical foundation the reader needs before encountering NeCT and the proposed extensions.]

2.1 Neural Networks

A neural network is a parameterised function whose internal parameters are adjusted iteratively until the function approximates a desired input-output relationship. The basic computational unit is the neuron, which computes a weighted sum of its inputs and passes the result through a nonlinear activation function. By stacking many neurons into layers and connecting layers sequentially, a network can represent increasingly complex relationships between its inputs and outputs.

The fundamental architecture used throughout this work is the *multilayer perceptron* (MLP), sometimes called a fully-connected or feedforward network. An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer ℓ applies a learned affine transformation followed by a nonlinear activation function:

$$\mathbf{h}^{(\ell)} = \sigma\left(W^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \quad (2.1)$$

where $W^{(\ell)}$ is the weight matrix, $\mathbf{b}^{(\ell)}$ is the bias vector of layer ℓ , and σ denotes the activation function applied element-wise. The input to the first layer is the network input $\mathbf{h}^{(0)} = \mathbf{x}$, and the output of the final layer L is the prediction $\hat{y} = \mathbf{h}^{(L)}$.

The composition of L such layers defines a function $f_{\theta} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, where θ collects all learnable parameters $\{W^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L$. By the universal approximation theorem, an MLP with at least one hidden layer and a nonlinear activation can approximate any continuous function on a compact domain to arbitrary precision given sufficient capacity [Cybenko, 1989, Hornik et al., 1989]. In practice, depth is often more parameter-efficient than width, which is why most modern MLPs use several moderately wide hidden layers rather than one very wide one.

The choice of activation function has a significant effect on training dynamics. The historically popular sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$ and hyperbolic tangent $\sigma(z) = \tanh(z)$ saturate for large input magnitudes, causing vanishing gradients in deep networks. The rectified linear unit (ReLU), $\sigma(z) = \max(0, z)$, largely solved this problem and became the standard choice for many years [Nair and Hinton, 2010]. A known failure mode of ReLU is that neurons whose pre-activation is consistently negative receive no gradient and become permanently inactive. The Leaky ReLU addresses this by allowing a small gradient for negative inputs:

$$\sigma(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0, \end{cases} \quad (2.2)$$

where $\alpha \ll 1$ is a small constant, commonly 0.01. For applications requiring the network to represent signals with significant high-frequency content, periodic activations such as sinusoidal representation networks (SIREN, $\sigma(z) = \sin(z)$) have been proposed [Sitzmann et al., 2020]. These can represent sharp features without any additional input encoding but are sensitive to initialisation and can be difficult to train stably.

Training a network means finding the parameter vector θ that minimises a scalar loss function $\mathcal{L}(\theta)$ measuring the discrepancy between predictions and observations. For regression tasks the two most common choices are the mean squared error (L2 loss) and the mean absolute error (L1 loss):

$$\mathcal{L}_{\text{L2}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad (2.3)$$

$$\mathcal{L}_{\text{L1}} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|. \quad (2.4)$$

L1 loss is less sensitive to outliers than L2 and is often preferred when the measurements contain occasional corrupted samples. Parameters are updated by gradient descent: at each step θ is adjusted in the direction of steepest descent of $\mathcal{L}$. The gradient $\nabla_{\theta} \mathcal{L}$ is computed by backpropagation, which applies the chain rule layer by layer from the output back to the input [Rumelhart et al., 1986]. In practice the gradient is estimated from a random mini-batch of B data points at each step rather than from the full dataset. The Adam optimiser [Kingma and Ba, 2015], which maintains per-parameter running estimates of the first and second gradient moments, is the standard choice for training coordinate networks.

2.1.1 Convolutional Neural Networks

While the MLP treats every input element independently, images and other spatially structured signals have strong local correlations: nearby pixels tend to be similar, and the same local pattern such as an edge or a boundary can appear at any position in the image. Convolutional neural networks (CNNs) exploit this structure by replacing the dense weight matrices of an MLP with learned local filters applied uniformly across the spatial extent of the input [LeCun et al., 1998].

A discrete 2D convolution of an input feature map $\mathbf{X} \in \mathbb{R}^{H \times W}$ with a filter $\mathbf{K} \in \mathbb{R}^{k \times k}$ produces an output feature map $\mathbf{Y}$ whose entries are

$$Y_{i,j} = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} K_{p,q} X_{i+p,j+q}. \quad (2.5)$$

The same kernel $\mathbf{K}$ is applied at every spatial position (i, j) , so the number of learnable parameters depends only on the kernel size k rather than the image dimensions. This weight sharing gives CNNs translation equivariance: if the input is shifted spatially, the output shifts by the same amount. Each output unit depends on only a $k \times k$ neighbourhood of the input, called its receptive field. Stacking multiple convolutional layers increases the effective receptive field of deeper units geometrically, allowing the network to integrate local evidence across progressively larger regions. Pooling layers, most commonly max-pooling, spatially downsample feature maps and introduce a degree of translation invariance beyond the equivariance of the convolution itself.

For CT image reconstruction, CNNs are typically applied in an image-to-image fashion: a reconstruction degraded by sparse-view artefacts, obtained for example from a fast analytical reconstruction, is passed through the network, which is trained to output a cleaner image [Jin et al., 2017]. The U-Net architecture [Ronneberger et al., 2015] has become dominant for this task. It consists of a contracting encoder path that progressively halves spatial resolution while doubling the number of feature channels, and a symmetric expanding decoder path that restores the original resolution using transposed convolutions. Skip connections concatenate feature maps from the encoder to the corresponding decoder stage, allowing the network to recover fine spatial detail that would otherwise be lost during downsampling. CNNs have also been applied directly in the sinogram domain, where a network learns to predict missing projections in a sparse-view sinogram before passing the completed sinogram to a conventional reconstruction algorithm [Hu et al., 2021]. Generative adversarial networks, which couple a generator network with an adversarially trained discriminator, have further improved the perceptual quality of CNN-based CT reconstructions [Goodfellow et al., 2014, Guo et al., 2022].

Despite their strong empirical performance on static image reconstruction tasks,

CNN-based approaches carry a set of structural limitations. A CNN must be trained on a dataset of paired examples before it can be deployed, so assembling training data for a new experimental configuration is expensive and time-consuming. Once trained, the network is committed to that configuration; applying it to a new one requires retraining. CNNs have no internal model of the measurement process and learn only a statistical mapping from the training distribution. When the test image differs from that distribution, as is common in novel scientific experiments, the network can confidently produce physically incorrect features [Patwari et al., 2023]. CNNs also operate on discrete pixel or voxel grids, so the spatial resolution is fixed at training time. Representing a continuously evolving three-dimensional object at arbitrary spatial and temporal coordinates is not a natural fit for the CNN framework, and constructing a four-dimensional reconstruction requires either treating each time step independently or processing large four-dimensional volumes, neither of which is computationally attractive at the resolutions used in micro-CT.

2.1.2 Implicit Neural Representations

An implicit neural representation (INR), sometimes called a coordinate network or neural field, is a neural network trained to represent a signal as a continuous function of its coordinates. Formally, an INR is a parameterised function

$$f_{\theta} : \mathbb{R}^n \longrightarrow \mathbb{R}^m, \quad (2.6)$$

where θ denotes the network weights, the input is a coordinate vector such as a spatial position or a position-time pair, and the output is the signal value at that coordinate. The signal is implicit in the sense that it is never stored as a discrete grid of samples. Instead, the continuous function f_{θ} is the representation itself, and individual values are obtained by evaluating the network at any desired coordinate.

This contrasts with conventional discrete representations. A 3D volume stored as a voxel grid of resolution N^3 requires memory proportional to N^3 , growing rapidly with resolution: a 1000^3 volume at 32-bit precision consumes roughly 4 GB. An INR stores only its network weights, whose count is independent of the desired output resolution. The same network architecture can be queried at 128^3 or 1024^3 spatial positions without any change to its parameter count, making INRs inherently resolution-agnostic.

An INR is instance-specific: it is optimised from scratch for each individual signal, using only the measurements of that signal as supervision. No external dataset of prior examples is needed. The optimisation proceeds by defining a differentiable forward model that maps the network’s predicted field to the expected measurements, then minimising the discrepancy between those predictions and the

actual observations. Because the physics of the measurement process is embedded directly in the optimisation loop, the reconstruction is constrained to be consistent with the known forward model. This prevents the generation of features that cannot be explained by the measurements, which is a significant failure mode of trained CNN approaches [Patwari et al., 2023].

A naive INR, one in which an MLP receives raw coordinate inputs, struggles to represent signals with significant high-frequency content such as sharp material boundaries. This is a consequence of the spectral bias of neural networks: gradient-based training causes MLPs to learn low-frequency components of a target function far faster than high-frequency ones [Rahaman et al., 2019]. Analysed through the neural tangent kernel (NTK), a standard coordinate-based MLP corresponds to a kernel whose eigenvalue spectrum decays rapidly with frequency, which prevents convergence to high-frequency components within a practical number of optimisation steps [Tancik et al., 2020].

The standard remedy is to map the raw input coordinates $\mathbf{x} \in \mathbb{R}^n$ through a fixed positional encoding $\gamma : \mathbb{R}^n \rightarrow \mathbb{R}^d$ before passing them to the MLP, where $d \gg n$:

$$f_{\theta}(\mathbf{x}) = \text{MLP}_{\theta}(\gamma(\mathbf{x})). \quad (2.7)$$

The sinusoidal positional encoding popularised by NeRF [Mildenhall et al., 2021] maps each coordinate to a sequence of sines and cosines at exponentially increasing frequencies:

$$\gamma(\mathbf{x}) = (\sin(2^0\pi\mathbf{x}), \cos(2^0\pi\mathbf{x}), \dots, \sin(2^{L-1}\pi\mathbf{x}), \cos(2^{L-1}\pi\mathbf{x})), \quad (2.8)$$

where L is the number of frequency octaves. Tancik et al. [2020] showed using NTK theory that this encoding transforms the effective kernel into a stationary one with a tunable bandwidth, directly addressing spectral bias. An alternative is to use random Fourier features $\gamma(\mathbf{x}) = [\sin(\mathbf{B}\mathbf{x}), \cos(\mathbf{B}\mathbf{x})]$, where each row of $\mathbf{B}$ is sampled from a Gaussian $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ with bandwidth σ chosen to match the signal’s frequency content.

Both sinusoidal encodings and random Fourier features are fixed: their parameters are set before training and not updated by gradient descent. A more capable alternative is to learn the encoding jointly with the MLP. The multiresolution hash encoding of Müller et al. [2022] pursues this by storing trainable feature vectors in a set of hash tables indexed by the input coordinates at multiple spatial resolutions. This shifts much of the representational work from the MLP to the encoding tables, allowing a smaller and faster MLP to achieve equal or better reconstruction quality. The hash encoding is described in detail in section 2.1.3 following the introduction of NeRF.

2.1.3 Neural Radiance Fields (NeRF)

Neural Radiance Fields (NeRF), introduced by Mildenhall et al. [2021], demonstrated that a complex 3D scene could be fully represented by the weights of a small MLP trained on a set of 2D photographs taken from known camera positions, and that this representation was sufficient to synthesise photorealistic images from arbitrary new viewpoints. What made NeRF influential beyond its original application was the mathematical structure it established: a differentiable forward model connecting a continuous volumetric field to 2D observations via a ray integral, trained end-to-end by minimising the discrepancy between simulated and measured projections. This same structure, a continuous scalar field recovered through a physics-based differentiable renderer, is directly applicable to tomographic reconstruction.

Scene Representation

NeRF represents a static scene as a continuous 5D function

$$F_{\Theta} : (\mathbf{x}, \mathbf{d}) \longrightarrow (\mathbf{c}, \sigma), \quad (2.9)$$

where $\mathbf{x} = (x, y, z) \in \mathbb{R}^3$ is a spatial location, $\mathbf{d} \in \mathbb{S}^2$ is a unit viewing direction, $\mathbf{c} = (r, g, b)$ is the emitted RGB colour at that location as seen from direction $\mathbf{d}$, and $\sigma \geq 0$ is the volume density at $\mathbf{x}$. The density $\sigma(\mathbf{x})$ represents the differential probability that a ray is absorbed at $\mathbf{x}$ and is treated as a property of the scene geometry alone, predicted from position only to enforce multi-view consistency. The colour $\mathbf{c}$ depends on both position and direction, allowing NeRF to represent view-dependent effects such as specular reflections.

In practice F_{Θ} is implemented as an MLP. The spatial coordinate $\mathbf{x}$ is first lifted by the sinusoidal positional encoding and passed through eight fully-connected ReLU layers with 256 channels each, producing σ and a 256-dimensional feature vector. The feature vector is then combined with the encoded viewing direction and passed through one further layer to produce $\mathbf{c}$ [Mildenhall et al., 2021]. Geometry is processed before appearance, which stabilises training by preventing the network from compensating for geometric errors through view-dependent colour.

To render the colour at a pixel, a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is cast from the camera origin $\mathbf{o}$ into the scene. The expected colour $C(\mathbf{r})$ accumulated along the ray between near and far bounds t_n and t_f is given by the classical volume rendering integral [Kajiya and Von Herzen, 1984]:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \quad (2.10)$$

where the accumulated transmittance $T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s))ds\right)$ is the probability that the ray travels from t_n to t without being absorbed. The integral is approximated using stratified sampling: the interval $[t_n, t_f]$ is divided into N equal bins and one sample t_i is drawn uniformly from each [Mildenhall et al., 2021]. The discrete approximation is

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i \left(1 - e^{-\sigma_i \delta_i}\right) \mathbf{c}_i, \quad T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right), \quad (2.11)$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples. This expression is fully differentiable with respect to σ_i and $\mathbf{c}_i$, and therefore with respect to the MLP parameters Θ .

NeRF is trained by minimising the squared photometric error between the rendered colour $\hat{C}(\mathbf{r})$ and the observed pixel colour $C(\mathbf{r})$ over all training rays. No 3D supervision is required; the only inputs are the 2D images and the known camera poses. NeRF additionally uses a hierarchical sampling strategy in which a coarse network places samples uniformly and a fine network concentrates additional samples in regions of high density [Mildenhall et al., 2021].

The structural parallel with CT reconstruction is close. In both settings a continuous 3D field is observed indirectly through line integrals, and the goal is to recover the field from a finite collection of such measurements. Table 2.1 summarises the correspondences.

Table 2.1: Structural analogy between NeRF and attenuation-contrast CT.

Concept	NeRF	CT
Recovered field	Volume density $\sigma(\mathbf{x})$	Attenuation coefficient $\mu(\mathbf{x})$
Measurements	Pixel colours $C(\mathbf{r})$	Detector intensities I_j
Forward model	Volume rendering integral	Beer–Lambert law
Ray sampling	Stratified random sampling	Equidistant ray marching
Loss function	L2 photometric error	L1 intensity error
Input encoding	Sinusoidal $\gamma(\mathbf{x})$	Multiresolution hash grid

The key difference between the two forward models is that in NeRF the transmittance is weighted by emitted colour, whereas in monochromatic CT no colour is emitted and the measured intensity follows Beer–Lambert’s law directly:

$$I_j = I_0 \exp\left(-\int_{\text{ray}_j} \mu(\mathbf{r}(t)) dt\right). \quad (2.12)$$

Setting $\sigma(\mathbf{x}) \equiv \mu(\mathbf{x})$ and dropping the colour output reduces the NeRF forward model to exactly this expression. The CT reconstruction problem can therefore be addressed using the same INR machinery as NeRF, substituting the Beer–Lambert

integral for the volume renderer and choosing an encoding suited to the properties of tomographic data.

Multiresolution Hash Encoding

The sinusoidal positional encoding resolves the spectral bias problem but places all of the representational work on the MLP: every gradient step updates every weight in the network, and convergence is slow regardless of how well the encoding lifts the input coordinates. The multiresolution hash encoding of Müller et al. [2022] addresses this by distributing the work between the MLP and a set of small trainable lookup tables, allowing the MLP itself to be made much smaller. The result is a speedup of several orders of magnitude over sinusoidal-encoded NeRF at equal or better reconstruction quality [Müller et al., 2022].

The encoding maintains L independent levels, each corresponding to a different spatial resolution. At each level ℓ a grid of resolution N_ℓ is defined, where the resolutions grow as a geometric progression between a coarsest resolution $N_{\min}$ and a finest resolution $N_{\max}$:

$$N_\ell = \lfloor N_{\min} \cdot b^\ell \rfloor, \quad b = \exp\left(\frac{\ln N_{\max} - \ln N_{\min}}{L - 1}\right). \quad (2.13)$$

Each level stores up to T trainable feature vectors of dimension F , arranged in a flat array. The level’s grid vertices are mapped into this array in one of two ways depending on the level’s resolution. At coarse levels, where the grid has fewer than T vertices, each vertex maps to a unique array entry (a 1:1 mapping, equivalent to a dense grid). At finer levels, where the number of grid vertices exceeds T , the array is treated as a *hash table* and each vertex is mapped to an entry by a spatial hash function $h : \mathbb{Z}^d \rightarrow \mathbb{Z}_T$. The total number of trainable encoding parameters is therefore bounded by $T \cdot L \cdot F$, regardless of the finest resolution $N_{\max}$, making the memory cost a controlled hyperparameter rather than a consequence of resolution. Typical values used in NECT are $L = 16$ levels, $T = 2^{23}$ entries, $F = 2$ features per entry, $N_{\min} = 16$, and a finest resolution calibrated to the voxel size of the CT scan [Friis et al., 2025].

Feature lookup and interpolation. Given an input coordinate $\mathbf{x} \in \mathbb{R}^d$, the encoding proceeds independently at each level ℓ as follows. First, $\mathbf{x}$ is scaled by the level’s grid resolution to find the enclosing voxel: the 2^d integer corner vertices of this voxel are identified. Each corner is mapped to an array index via either the 1:1 mapping (coarse levels) or the hash function (fine levels), and the corresponding F -dimensional feature vectors are retrieved. These 2^d feature vectors are then *linearly interpolated* according to the fractional position of $\mathbf{x}$ within its enclosing voxel —

trilinear interpolation in 3D, or the 4D analogue in space-time. The interpolated F -dimensional vector is the output of level ℓ . The outputs from all L levels are *concatenated* to form the full encoded representation $\mathbf{y} \in \mathbb{R}^{L \cdot F}$, which is then passed to the MLP:

$$\hat{\mu}(\mathbf{x}) = \text{MLP}_{\Phi}(\mathbf{y}(\mathbf{x}; \theta)), \quad \mathbf{y} = \left[\text{interp}_0(\mathbf{x}; \theta_0), \text{interp}_1(\mathbf{x}; \theta_1), \dots, \text{interp}_{L-1}(\mathbf{x}; \theta_{L-1}) \right]. \quad (2.14)$$

Crucially, linear interpolation is differentiable, so gradients flow back through the concatenation and interpolation to the feature vectors θ_ℓ at each level, updating only the 2^d entries involved in each forward pass. For a 3D grid this is 8 entries per level per sample rather than every weight in the MLP. This sparse gradient update is the primary reason the encoding converges so much faster than a purely MLP-based representation [Müller et al., 2022].

At fine resolution levels, multiple grid vertices inevitably map to the same hash table entry. Unlike classical hash tables which resolve such collisions through chaining or probing, the multiresolution hash encoding handles them implicitly. When two vertices collide, their gradients accumulate in the same feature vector during training. The gradients from the region with the highest loss dominate the update, so the shared vector is pulled toward representing the most important detail. The MLP then uses context from the other $L - 1$ levels, which encode the same location at different resolutions with different hash functions, to disambiguate colliding entries at query time. In practice collisions are most frequent at the finest levels, where they concentrate representational capacity on regions of high spatial detail. No explicit data-structure updates such as pruning or splitting are needed at any point during training [Müller et al., 2022], which maps well to GPU parallelism.

Because hash table lookups are $\mathcal{O}(1)$ with no control-flow branching, all L levels can be evaluated in parallel on a GPU. The small MLP that follows requires far fewer floating-point operations than the large MLP needed by sinusoidal encodings. Together these properties enable training to convergence in seconds rather than hours on a single GPU, a speedup of roughly two to three orders of magnitude over the original NeRF implementation [Müller et al., 2022].

The first application of multiresolution hash encoding to CT reconstruction was Neural Attenuation Fields (NAF) by Zha et al. [2022]. NAF parameterises the 3D attenuation coefficient field $\mu(\mathbf{x})$ as a hash-encoded MLP and trains the network by minimising the discrepancy between simulated and measured cone-beam projections using Beer–Lambert’s law as the forward model, with no external training data required. NAF showed that the hash encoding’s advantages in NeRF, faster convergence and better recovery of high-frequency detail, transfer directly to CT and outperform both sinusoidal-encoded INRs and iterative algorithms at a fraction of the

computational cost [Zha et al., 2022]. The physical motivation is that CT attenuation fields are typically smooth within homogeneous material phases but sharp at phase boundaries. Coarse hash levels capture the smooth large-scale variation efficiently while fine levels concentrate capacity at the boundaries where the self-disambiguation mechanism is most useful.

2.2 X-ray Computed Tomography

2.2.1 Physical Principles

X-ray computed tomography reconstructs the three-dimensional internal structure of an object from a set of its two-dimensional projections. The physical quantity being reconstructed is the linear attenuation coefficient $\mu(\mathbf{r})$, a material-dependent scalar field defined at every point $\mathbf{r} = (x, y, z)$ in the object that describes how strongly the local material absorbs or scatters X-ray photons. Its SI unit is m^{-1} and it depends on both the material density and the photon energy. In attenuation-contrast CT the contrast between different materials arises entirely from differences in μ : dense materials such as rock matrix and precipitated salt have high μ , air-filled pores have low μ , and brine lies in between.

When a monochromatic X-ray beam of initial intensity I_0 travels through a heterogeneous material along a straight ray path, the transmitted intensity I is governed by Beer–Lambert’s law:

$$I = I_0 \exp\left(-\int_{\text{ray}} \mu(\mathbf{r}(t)) dt\right), \quad (2.15)$$

where the integral is taken along the parameterised ray path $\mathbf{r}(t) = \mathbf{o} + t \mathbf{d}$, with $\mathbf{o}$ the source position and $\mathbf{d}$ a unit direction vector. Taking the negative logarithm of the transmitted-to-incident intensity ratio gives the line integral of μ along the ray, also called the projection value p :

$$p = -\ln\left(\frac{I}{I_0}\right) = \int_{\text{ray}} \mu(\mathbf{r}(t)) dt. \quad (2.16)$$

CT reconstruction is the inverse problem of recovering $\mu(\mathbf{r})$ from a finite set of such projection values, each corresponding to a different ray direction and detector pixel. Beer–Lambert’s law assumes the X-ray beam is effectively monochromatic and that scattering is negligible; both are standard assumptions in laboratory micro-CT analysis [Kak and Slaney, 2001].

In cone-beam CT (CBCT), a point X-ray source illuminates the entire sample simultaneously and the transmitted intensities are recorded on a flat 2D area detec-

tor. This is the geometry of standard laboratory micro-CT instruments. A single projection, also called a radiograph, is the 2D image $\{I_j\}$ recorded by the detector at one angular position of the rotating sample, where j indexes the detector pixels. The source-to-detector distance and source-to-object distance together determine the geometric magnification and hence the effective voxel size of the reconstruction. As the sample rotates, successive projections sweep a full 360° arc, and the complete set of projections forms the sinogram, a 3D data volume whose dimensions are the number of angles, and the two detector pixel dimensions. Each row of a 2D sinogram slice traces a sinusoidal curve as the angular position varies, which is the origin of the name.

In practice the continuous Beer–Lambert integral is approximated by a discrete sum over K sample points along the ray:

$$p_j \approx \sum_{k=1}^K \mu(\mathbf{r}_k) \Delta t, \quad (2.17)$$

where Δt is the sampling step size. CT reconstruction then amounts to inverting the system of equations $\{p_j = \mathcal{A}\mu\}$, where $\mathcal{A}$ is the forward projection operator that maps the attenuation field to the measured projection values. The system is highly underdetermined when the number of projections is small relative to the number of voxels, making the inverse problem ill-posed and requiring regularisation or prior information for a stable solution.

2.2.2 Classical Reconstruction Algorithms

A large body of reconstruction algorithms has been developed to invert the forward projection operator $\mathcal{A}$ from eq. (2.17). They fall into two broad families: *analytical* methods, which derive a closed-form inversion formula from the properties of the Radon transform, and *iterative* methods, which formulate reconstruction as an optimisation problem and solve it by successive approximation. The three algorithms used as baselines in NECT — FDK, OS-SART-TV, and, implicitly, filtered back-projection — span both families and are described here.

Filtered Back-Projection and FDK

The analytical foundation of most classical CT reconstruction is the *filtered back-projection* (FBP) algorithm, which follows directly from the *Fourier slice theorem*: the 1D Fourier transform of a parallel-beam projection at angle ϕ is a slice through the 2D Fourier transform of $\mu(\mathbf{r})$ at the same angle. A full set of projections covering 180° therefore provides complete coverage of the 2D Fourier domain, and μ can be recovered by an inverse 2D Fourier transform. In the spatial domain this is equivalent

to back-projecting each filtered projection onto a reconstruction grid. The ramp filter applied before back-projection compensates for the non-uniform polar sampling of the Fourier domain, which would otherwise over-represent low spatial frequencies and produce blurred reconstructions [Kak and Slaney, 2001].

The Feldkamp–Davis–Kress (FDK) algorithm [Feldkamp et al., 1984] is the cone-beam extension of FBP. It approximates the full 3D cone-beam geometry by treating each detector row as an independent tilted fan-beam slice, applies a cosine-weighted ramp filter to each row of each projection, and back-projects the filtered data into a 3D reconstruction volume. For small cone angles this approximation is accurate, and FDK is accepted as the standard workhorse for laboratory CBCT because it is non-iterative and extremely fast [Kak and Slaney, 2001].

FDK’s critical requirement is that the number of projections N_ϕ must satisfy the Shannon–Nyquist criterion, which for a reconstruction of N pixels across the field of view demands approximately $N_\phi \geq \pi N/2$ projections to avoid aliasing artefacts [Kak and Slaney, 2001]. When this condition is met, FDK reconstructions are of high quality. When it is violated, as it inevitably is in fast or sparse-view CT, the reconstructions suffer from characteristic streak artefacts that radiate from high-contrast features and obscure fine pore-level detail.

Iterative methods formulate reconstruction as the minimisation of a data fidelity term measuring the discrepancy between the forward-projected estimate and the measured projections, optionally supplemented by a regularisation term. The basic algebraic reconstruction technique (ART) updates the estimate ray by ray [Gordon et al., 1970]. The simultaneous algebraic reconstruction technique (SART) [Andersen and Kak, 1984] improves convergence by updating all voxels simultaneously using rays from one projection angle at each step. Ordered subsets SART (OS-SART) further accelerates convergence by processing projections in angularly diverse subsets, so each subset update provides approximately as much new information as a full SART iteration [Du et al., 2017].

Adding a total variation (TV) regularisation term to the OS-SART objective yields OS-SART-TV, which is one of the main baselines used for comparison in the NeCT paper. TV regularisation penalises the sum of the magnitudes of the image gradient across all voxels:

$$\mathcal{R}_{\text{TV}}(\mu) = \sum_{\mathbf{r}} \|\nabla \mu(\mathbf{r})\|_2, \quad (2.18)$$

which promotes piecewise-constant solutions, images with flat homogeneous regions separated by sharp boundaries. This is physically motivated for porous media where the attenuation coefficient is approximately uniform within each material phase. TV regularisation can substantially suppress streak artefacts compared to FDK in

sparse-view conditions, but it introduces contrast reduction: the penalty discourages gradients everywhere, so small features and fine pore structures tend to be smoothed out and the reconstructed values within homogeneous regions are pulled toward a common mean [Kak and Slaney, 2001].

Both FDK and OS-SART-TV were designed for fully-sampled static 3D CT, and their limitations become severe in sparse-view and 4D settings. The most direct approach to 4D-CT is to acquire a sequence of independent full 3D scans, reconstructing each with FDK. The temporal resolution is then equal to the time required to complete one full rotation, typically around one hour for laboratory micro-CT, making it impossible to observe fast events that unfold on timescales of seconds to minutes. Reducing the number of projections per time step improves temporal resolution but causes severe streak artefacts in FDK and contrast loss in OS-SART-TV. Neither algorithm can make use of information from other time steps to inform a given frame because each reconstruction is treated independently. Both methods also produce a discrete sequence of independent volumetric images with no model of the object’s temporal evolution, so no information is shared across frames.

2.2.3 4D-CT and Dynamic Imaging

4D-CT denotes the acquisition and reconstruction of CT data from a sample that evolves over time, producing a time-resolved sequence of 3D images. The fourth dimension is time, and the full dataset can be thought of as a continuous field $\mu(\mathbf{r}, t)$ sampled at a finite set of spatial voxels and time steps. In geosciences, 4D-CT is applied to study mechanical deformation of rocks, multiphase fluid flow in porous media, CO₂ mineralisation under reservoir conditions, and reactive dissolution of mineral grains [Withers et al., 2021]. The central challenge in all these applications is achieving sufficient temporal resolution to capture the process of interest while maintaining the spatial resolution needed to observe pore-scale phenomena.

The temporal resolution of a 4D-CT scan is ultimately limited by the time required to collect enough angular projections to reconstruct a 3D volume. In conventional sequential scanning a full 360° rotation is completed before moving to the next time step, giving a temporal resolution on the order of one hour for laboratory micro-CT. Reducing this requires acquiring far fewer projections per time step. The resulting trade-off is that temporal resolution improves as the number of projections per frame decreases, but reconstruction quality degrades as the inverse problem becomes increasingly underdetermined. Several approaches have been developed to mitigate this. One class of methods incorporates prior information into the reconstruction, for example using a high-quality static scan as a reference frame [Chen et al., 2008, Myers et al., 2011]. Another class uses regularisation tailored to the expected temporal

evolution, such as penalising the temporal derivative of μ to enforce smooth changes over time [Goethals et al., 2022]. A third class represents the object as a continuous function of space and time, fitting a single model to all projections from the entire experiment.

The photon flux of the X-ray source is a fundamental constraint on temporal resolution. Synchrotron radiation sources of the third and fourth generation provide a beam brilliance 10–14 orders of magnitude higher than laboratory X-ray tubes [Withers et al., 2021], enabling sub-second exposures and hundreds of projections per second. Sub-second temporal resolution is therefore achievable at synchrotrons, where 4D micro-CT has been applied to phenomena such as drainage dynamics and insect flight mechanics. Standard laboratory micro-CT instruments are far more accessible but each radiograph requires a finite exposure time to achieve an acceptable signal-to-noise ratio, which limits acquisition speed.

The choice of angular sampling scheme significantly affects how much complementary information is gathered per projection in a sparse-view acquisition. In conventional equiangular scanning, projections are collected at equally spaced angles around 360° . When a full rotation is completed before beginning the next, this is well-suited to static reconstruction but produces temporal aliasing in dynamic experiments: consecutive projections are taken at similar angles, providing redundant information about the current state of the sample.

The golden-angle sampling scheme addresses this by setting the angular increment between successive projections to $\Delta\phi \approx 137.51^\circ$, which is the angle that most uniformly distributes projections on a circle as more are accumulated [Kohler, 2004]. At any point during the acquisition the projections so far are nearly optimally spread in angle, so a reconstruction from the first N projections is well-conditioned regardless of the total number eventually collected. This property is useful for time-resolved imaging where the effective number of projections per time step is small.

A practical limitation of the pure golden-angle scheme is that consecutive projections are separated by 137.51° , requiring the sample stage to travel through a large angle between exposures. In laboratory instruments this large step is time-consuming and can cause vibration. The hybrid golden-section scheme used in NeCT experiments [Friis et al., 2025] addresses this: within each full 360° revolution a fixed number of equidistant projections are collected, but the starting angle of each revolution is offset from the previous one by the golden angle. This preserves rapid continuous rotation within each revolution while ensuring that successive revolutions are angularly offset, so that projections from different time points together cover the angular space as uniformly as possible.

2.3 NeCT: Neural Computed Tomography

NeCT (Neural Computed Tomography) [Friis et al., 2025] is, to the best of the authors’ knowledge, the first fully INR-based algorithm for holistic time-resolved 4D-CT reconstruction. It unifies the physical forward model of cone-beam CT with the representational power of hash-encoded implicit neural representations, treating the entire dynamic experiment, all projections from all time steps, as a single joint optimisation problem. The output is a continuous function $\hat{\mu}(\mathbf{r}, t)$, the estimated attenuation coefficient at any spatial coordinate $\mathbf{r}$ and any time t within the duration of the scan, from which volumetric images at arbitrary time points can be decoded by a forward pass through the network.

NeCT consists of two architecturally related but distinct models: a static 3D model for sparse-view single-frame CT reconstruction (an improved version of NAF [Zha et al., 2022]), and a dynamic 4D model called *NeCT QuadCubes* for time-resolved reconstruction. Both share the same training loop and loss function; they differ in the structure of the hash encoding and whether the time coordinate t is included as an input. The static model serves as a benchmark for spatial reconstruction quality, while the dynamic model is the primary contribution relevant to the extensions developed in this thesis.

2.3.1 NeCT Pipeline Overview

The NeCT pipeline operates as a differentiable physics simulation. It takes the measured detector images as supervision and adjusts the network weights until the network’s predictions are consistent with every recorded projection. The same four-step loop repeats for every training batch.

At the start of each step, a batch of B rays is sampled from the full set of recorded projections. Each ray $\mathbf{r}_j(t) = \mathbf{o}_j + t \mathbf{d}_j$ is defined by its source position $\mathbf{o}_j$ and unit direction $\mathbf{d}_j$, determined by the cone-beam geometry of the instrument and the angular position of the projection from which the ray originates. In the dynamic model, each ray also carries a timestamp t_j corresponding to the acquisition time of its projection. The measured intensity I_j at the corresponding detector pixel is retrieved as the supervision target.

Each ray is then marched through the object volume by sampling K equidistant points $\{\mathbf{r}_k\}_{k=1}^K$ along the ray within the sample bounding volume. The number of sample points per ray is not fixed; NeCT uses an adaptive scheme in which K increases linearly from $K_{\text{init}} = 100$ to $K_{\text{max}} = 1.5 \times \max(N_{\text{detector}})$ over the first 30% of training, where $\max(N_{\text{detector}})$ is the number of pixels along the longest detector axis [Friis et al., 2025]. Starting with fewer points reduces computational cost in the early stages when the network is still poorly initialised, and progressively

increasing the count ensures the final reconstruction is sampled finely enough to resolve sub-voxel features.

Each sampled point $(\mathbf{r}_k, t_j)$ is passed through the neural network to obtain the predicted attenuation coefficient $\hat{\mu}(\mathbf{r}_k, t_j)$. In the static model t_j is omitted. The predicted detector intensity for ray j is then computed by discretising Beer–Lambert’s law using the midpoint rule:

$$\hat{I}_j = I_0 \exp\left(-\sum_{k=1}^K \hat{\mu}(\mathbf{r}_k, t_j) \Delta s\right), \quad (2.19)$$

where Δs is the step size between adjacent sample points. The exponential is computed from the summed attenuation rather than as a product of per-step transmittances, which is numerically equivalent under the midpoint approximation and more efficient to evaluate.

The predicted intensities $\hat{I}_j$ are compared to the measured intensities I_j using the L1 loss over the batch:

$$\mathcal{L} = \frac{1}{B} \sum_{j=1}^B |\hat{I}_j - I_j|. \quad (2.20)$$

Gradients of $\mathcal{L}$ with respect to all network parameters (Φ, θ) , the MLP weights Φ and the hash table feature vectors θ , are computed by automatic differentiation and used to update the parameters via the Adam optimiser [Kingma and Ba, 2015]. The learning rate follows a schedule with a linear warm-up over the first 5 000 batches, after which it decays according to a cosine schedule to 1% of its peak value [Friis et al., 2025]. Base learning rates are 1×10^{-3} for static reconstructions and 2×10^{-4} for dynamic reconstructions. The batch size is 5×10^6 ray sample points per step.

This loop repeats for all training epochs. Because the hash table entries are updated only for the small subset of grid vertices intersected by each batch of rays, just $2^3 = 8$ entries per level per ray for the 3D encoder, the effective gradient update is extremely sparse, which is what gives the hash encoding its computational efficiency.

2.3.2 Static 3D Reconstruction: Improved NAF

The static NeCT model represents the attenuation coefficient as a continuous function of spatial coordinates only, $\hat{\mu} : \mathbb{R}^3 \rightarrow \mathbb{R}$, implemented as a hash-encoded MLP. It is architecturally derived from NAF [Zha et al., 2022] but with a refined encoder configuration and training protocol, and was benchmarked against FDK, OS-SART-TV, and NAF on eleven standard 3D CT datasets from the Open SciVis Datasets project [Klacansky, 2017].

The input spatial coordinate $\mathbf{r} = (x, y, z)$ is passed through a single 3D multiresolution hash encoder. The encoder uses a hashmap with $T = 2^{23}$ entries, $L = 16$

resolution levels, a base resolution of $N_{\min} = 16$, and $F = 4$ features per entry, yielding a 64-dimensional concatenated feature vector [Friis et al., 2025]. The MLP that follows has 4 hidden layers each with 128 neurons and Leaky ReLU activations, with a single linear output neuron producing the scalar $\hat{\mu}(\mathbf{r})$. The total number of trainable parameters is dominated by the hash table, roughly 537 million encoding parameters, compared to a few tens of thousands of MLP weights. In practice only the entries intersected by training rays are ever updated, so the effective active parameter count is much lower. The finest grid resolution $N_{\max}$ is calibrated to the detector resolution of the specific CT scan so that the finest hash grid voxels correspond approximately to the physical voxel size of the reconstruction.

Reconstructions were produced from 49 projections for each dataset, far below the Shannon–Nyquist requirement for the object sizes involved. NeCT was evaluated against FDK, OS-SART-TV, and NAF using peak signal-to-noise ratio (PSNR) and the structural similarity index measure (SSIM) [Wang et al., 2004]. PSNR is defined in terms of the mean squared error (MSE) between the reconstructed image $\hat{\mu}$ and the ground truth μ^* , normalised by the dynamic range R :

$$\text{PSNR} = 10 \log_{10} \left(\frac{R^2}{\text{MSE}(\hat{\mu}, \mu^*)} \right), \quad (2.21)$$

measured in decibels. SSIM captures perceptual similarity by comparing local statistics of luminance, contrast, and structure; it ranges from 0 to 1 [Wang et al., 2004]. Both metrics are computed volumetrically on the full 3D reconstruction.

NeCT outperformed all three competing methods on both metrics across all eleven datasets [Friis et al., 2025]. On the Stagbeetle dataset, representative of a high-complexity biological specimen, NeCT achieved a PSNR of 45.6 dB and SSIM of 0.996, compared to 44.3 dB / 0.994 for NAF, 41.9 dB / 0.990 for OS-SART-TV, and 28.6 dB / 0.577 for FDK. The improvement over NAF reflects the refined encoder configuration and the extended training protocol. Visually, NeCT reconstructions are free of the streak artefacts that dominate FDK results and the contrast-reduced appearance of OS-SART-TV, showing sharp material boundaries with minimal spurious features. This clean appearance despite using a fraction of the projections required by FDK is attributed in part to the spectral regularisation effect of the hash-encoded INR, which is spectrally biased toward smooth solutions and suppresses high-frequency streak patterns while preserving sharp boundaries between material phases [Friis et al., 2025].

2.3.3 Dynamic 4D Reconstruction: NeCT QuadCubes

The dynamic NeCT model, named QuadCubes, extends the 3D INR to four dimensions by representing the attenuation field as a continuous function of both space and time, $\hat{\mu} : \mathbb{R}^3 \times \mathbb{R} \rightarrow \mathbb{R}$. The central design challenge is how to structure the hash encoding for a 4D input space.

The most straightforward extension would be a single 4D hash encoder indexing its tables with (x, y, z, t) corner vertices. This has two problems. A 4D voxel has $2^4 = 16$ corner vertices rather than $2^3 = 8$, doubling the number of table lookups and gradient updates per sample. More severely, the hash collision rate increases dramatically: for a fixed table size T , the expected number of 4D grid vertices per table entry at the finest level grows as $N_{\max}^4/T$, which at the resolutions required for micro-CT becomes large enough that the network’s collision disambiguation mechanism is overwhelmed. At the other extreme, one could use many low-dimensional encoders, for example six 2D encoders covering all coordinate pairs, as in the K-Planes [Fridovich-Keil et al., 2023] and HexPlane [Cao and Johnson, 2023] architectures. This avoids the collision problem but each encoder captures only pairwise dependencies, placing a heavy burden on the MLP to integrate information across all four dimensions.

QuadCubes is a compromise between these extremes [Friis et al., 2025]. It uses four independent 3D multiresolution hash encoders, each covering a different triplet of the four space-time dimensions:

$$\text{Encoder 1: } (x, y, z), \quad \text{Encoder 2: } (x, y, t), \quad \text{Encoder 3: } (x, z, t), \quad \text{Encoder 4: } (y, z, t). \quad (2.22)$$

This set covers all four possible 3-element subsets of $\{x, y, z, t\}$, so every pair of coordinates appears together in at least two encoders. Each encoder independently produces a feature vector of dimension $L \cdot F$, and the four vectors are concatenated to form the full MLP input:

$$\hat{\mu}(\mathbf{r}, t) = \text{MLP}_{\Phi} \left(\left[\gamma_{xyz}(\mathbf{r}; \theta_1), \gamma_{xyt}(x, y, t; \theta_2), \gamma_{xzt}(x, z, t; \theta_3), \gamma_{yzt}(y, z, t; \theta_4) \right] \right), \quad (2.23)$$

where $\gamma_{(\cdot)}$ denotes the trilinear-interpolated hash encoding in the indicated coordinate triplet and θ_i are the trainable feature vectors of encoder i . The MLP has the same architecture as the static model: 4 hidden layers of 128 neurons with Leaky ReLU activations.

Each encoder operates in a well-conditioned 3D hash space where the collision rate remains manageable at micro-CT resolutions. Each encoder also jointly represents three coordinates, so cross-dimensional interactions within each triplet are captured directly in the feature space rather than delegated entirely to the MLP. The overlap between encoders, where the spatial encoder **xyz** shares each of its three

dimensions with one of the mixed encoders, provides redundant representations of spatial structure that assist collision disambiguation and give the MLP multiple views of the same location at different times.

A defining property of the QuadCubes model is that it is trained on all projections from the full dynamic experiment simultaneously rather than reconstructing each time step independently. All N_ϕ projections collected over the entire experiment contribute gradients to the same set of network parameters at every training step. The network learns to distinguish between time-invariant structural features of the rock matrix, which are supported by projections from all angles and all times, and dynamic features such as advancing fluid fronts or dissolving grains, which are supported only by projections in their temporal vicinity. Static structures therefore converge to a sharp well-defined representation, while dynamic features converge more slowly with spatial precision proportional to the rate at which they evolve relative to the projection acquisition rate. The temporal resolution of QuadCubes is consequently scene-dependent: for large, rapidly moving features a temporal resolution approaching the time between two successive projections is achievable [Friis et al., 2025].

Simulations using synthetic 4D datasets of liquid invasion in porous media generated with PoreSpy [Gostick et al., 2019] showed that QuadCubes can resolve dynamics lasting as few as two projection intervals for large features, and approaches full resolution for events spanning ten or more projections. In the experimental Bentheimer sandstone dataset, individual pore-filling events with a characteristic filling time of approximately 10 s and salt dissolution events lasting approximately 30 s were directly observed and quantified [Friis et al., 2025].

2.3.4 Hybrid Golden-Section Sampling

The hybrid golden-section sampling scheme was introduced in the context of 4D-CT acquisition in section 2.2.3. From the perspective of the reconstruction algorithm it is worth examining more closely, because the sampling strategy and the QuadCubes model are designed to work together.

The scheme provides what can be called progressive angular completeness: at any point in the experiment, the projections collected so far are as uniformly distributed in angle as possible. This ensures that the spatial encoder **xyz** receives gradient updates from many different projection directions simultaneously rather than from a narrow angular range, which would leave the spatial reconstruction poorly conditioned in early training. Without angular diversity, the model would converge slowly to an accurate representation of the static rock structure and temporal features would likewise be poorly resolved.

For the temporal encoders $\mathbf{xyt}$, $\mathbf{xzt}$, and $\mathbf{yzt}$, the golden offset between successive revolutions means that each new revolution contributes projections at angles that are maximally distinct from those of all previous revolutions. When a gradient update is computed for a ray from revolution k , the temporal encoders are updated at the time t_k of that revolution. Because each revolution samples an angularly offset range, successive updates to the temporal encoders carry complementary spatial information and together allow the model to distinguish the attenuation field at t_k from the field at neighbouring time steps.

In the Bentheimer sandstone experiment, 1 400 projections were distributed across 14 revolutions of 100 projections each, with the golden offset applied between revolutions and alternating rotation direction to avoid tangling the inlet supply lines [Friis et al., 2025]. The entire 40-minute imbibition experiment was represented by a single QuadCubes model, decoded at arbitrary times to produce a continuous 3D movie of the brine invasion process.

Chapter 3

Encoder Size and Performance

[TODO: Systematic study of how encoder capacity affects reconstruction quality and computational cost. All experiments use the same training protocol unless stated otherwise.]

3.1 Experimental Setup

[TODO: Datasets used (static benchmark + dynamic PoreSpy simulations), hardware (Nvidia A100/H100), metrics (PSNR, SSIM, MAE), training budget.]

3.2 Effect of Total Parameter Count

[TODO: Sweep encoder size (hashmap entries, number of levels, features per level, MLP width/depth). Plot PSNR/SSIM vs. number of parameters.]

3.3 Effect on Reconstruction Time

[TODO: Plot PSNR/SSIM vs. wall-clock training time. Identify the quality-vs-speed Pareto front.]

3.4 Precision and Accuracy Metrics

[TODO: Compare PSNR, SSIM, MAE, and optionally perceptual metrics. Discuss which metric best reflects physical fidelity for CT.]

3.5 Non-Uniform Encoder Sizing

[TODO: Motivation: spatial and temporal dimensions may benefit from different capacity allocations. Experiment with allocating more levels/features to the xyz encoder vs. temporal encoders. Compare against uniform baseline.]

Chapter 4

Encoder Architecture

[TODO: Investigate how the number and dimensionality of hash encoders affect reconstruction quality, hash collision rate, and coupling between dimensions.]

4.1 Motivation and Design Space

[TODO: Recall the original QuadCubes design rationale: balance between hash collisions (few high-dim encoders) and weak coupling (many low-dim encoders). Formalise the design space.]

4.2 QuadCubes (Baseline)

[TODO: Recap of the four 3D encoders (xyz, xyt, xzt, yzt) from NeCT. Serves as the baseline for this chapter.]

4.3 TriCubes

[TODO: Three 3D encoders. Which triplet of dimension combinations? Ablation results versus QuadCubes.]

4.4 SexCubes (Six Encoders)

[TODO: Six encoders covering all $\binom{4}{3} = 4$ triples plus additional combinations. Assess whether more encoders improve quality or just add cost.]

4.5 Single Encoder

[TODO: A single 4D hash encoder. Discuss hash collision severity in 4D and quality penalty compared to the split designs.]

4.6 Combined Cubes Method

[TODO: One 3D spatial encoder (xyz) combined with three 2D temporal slice encoders (xt, yt, zt). Motivation: preserve full spatial resolution while capturing temporal dynamics via lightweight 2D planes. Compare to QuadCubes in quality, speed, and parameter count.]

4.7 Comparative Analysis

[TODO: Summary table and plots. Discuss trade-offs: reconstruction quality, training time, memory footprint, and implementation complexity.]

Chapter 5

Continuous Scanning

[TODO: Extend NeCT to handle data acquired during continuous (fly-scan) rotation, where projection blur due to constant angular velocity must be accounted for.]

5.1 Motivation

[TODO: Explain motion smearing in continuous scanning and why ignoring it degrades reconstruction. Potential gain in temporal resolution.]

5.2 Forward Model for Continuous Scanning

[TODO: Derive the modified Beer–Lambert integral that averages attenuation over the angular interval $[\phi, \phi + \Delta\phi]$ swept during one exposure. Present the discrete approximation used in practice.]

Algorithm 1 Continuous-Scan Forward Pass in NECTv2

Require: Angular interval $[\phi_s, \phi_e]$, K sub-steps, ray $\mathbf{r}_j$

- 1: $\Delta\phi \leftarrow (\phi_e - \phi_s)/K$
 - 2: $\hat{I}_j \leftarrow 0$
 - 3: **for** $k = 0, \dots, K - 1$ **do**
 - 4: $\phi_k \leftarrow \phi_s + (k + 0.5) \Delta\phi$ ▷ Mid-point angle
 - 5: Sample points along ray $\mathbf{r}_j$ at angle ϕ_k
 - 6: $\hat{I}_j \leftarrow \hat{I}_j + \frac{1}{K} \exp\left(-\sum_p \hat{\mu}(\mathbf{r}_p, t) \Delta s\right)$
 - 7: **end for**
 - 8: **return** $\hat{I}_j$
-

5.3 Static Dataset Experiments

[TODO: Apply continuous-scan forward model to a static benchmark. Compare step-wise (non-continuous) reconstruction against continuous reconstruction with varying numbers of accumulation sub-steps K . Plot PSNR/SSIM vs. K .]

5.4 Dynamic Dataset Experiments

[TODO: Repeat the accumulation-step study on a dynamic PoreSpy simulation and/or the Bentheimer sandstone dataset. Assess whether the continuous model better captures fast pore-filling events.]

5.5 Discussion

[TODO: When does continuous scanning provide a meaningful benefit? Computational overhead vs. quality gain. Limitations.]

Chapter 6

Conclusion

6.1 Summary of Contributions

[TODO: Bullet-point recap of what was achieved in each chapter.]

6.2 Limitations and Future Work

[TODO: Remaining open questions: spatiotemporal resolution quantification, hyperparameter sensitivity, training time, extension to polychromatic sources, parallel-beam synchrotron settings, energy-resolved CT.]

Bibliography

- A. H. Andersen and A. C. Kak. Simultaneous algebraic reconstruction technique (SART): A superior implementation of the ART algorithm. *Ultrasonic Imaging*, 6(1):81–94, 1984. doi: 10.1177/016173468400600107.
- Ang Cao and Justin Johnson. HexPlane: A fast representation for dynamic scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 130–141, 2023.
- Guang-Hong Chen, Jie Tang, and Shuai Leng. Prior image constrained compressed sensing (PICCS): a method to accurately reconstruct dynamic CT images from highly undersampled projection data sets. *Medical Physics*, 35(2):660–663, 2008. doi: 10.1118/1.2836423.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.
- Yang Du, Gao Yu, Xia Xiang, Xiaoming Wang, and Yves De Deene. Convergence of SART + OS + TV iterative reconstruction algorithm for optical CT imaging of gel dosimeters. *Journal of Physics: Conference Series*, 847:012025, 2017. doi: 10.1088/1742-6596/847/1/012025.
- L. A. Feldkamp, L. C. Davis, and J. W. Kress. Practical cone-beam algorithm. *Journal of the Optical Society of America A*, 1(6):612–619, 1984. doi: 10.1364/JOSAA.1.000612.
- Sara Fridovich-Keil, Giacomo Meanti, Frederik Warburg, Benjamin Recht, and Angjoo Kanazawa. K-Planes: Explicit radiance fields in space, time, and appearance. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12479–12488, 2023.
- Henrik Friis, Håkon Nese, Giacomo Luani, Colin Pryme, Lars Rennan, Basab Chattopadhyay, Anders Kristoffersen, Ole Jakob Mengshoel, and Dag Werner Breiby. Implicit neural representation for fast 4D computed tomography of

- multiphase flow in porous media. *Communications Physics*, 8:339, 2025. doi: 10.1038/s42005-025-02249-0.
- Wout Goethals et al. Dynamic CT reconstruction with improved temporal resolution for scanning of fluid flow in porous media. *Water Resources Research*, 58:e2021WR031365, 2022. doi: 10.1029/2021WR031365.
- Ian Goodfellow et al. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, pages 2672–2680, 2014.
- Richard Gordon, Robert Bender, and Gabor T. Herman. Algebraic reconstruction techniques (ART) for three-dimensional electron microscopy and X-ray photography. *Journal of Theoretical Biology*, 29(3):471–481, 1970. doi: 10.1016/0022-5193(70)90109-8.
- Jeff Gostick et al. PoreSpy: A Python toolkit for quantitative analysis of porous media images. *Journal of Open Source Software*, 4(37):1296, 2019. doi: 10.21105/joss.01296.
- Ziyu Guo et al. Physics-assisted generative adversarial network for X-ray tomography. *Optics Express*, 30(13):23238–23259, 2022. doi: 10.1364/OE.459948.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.
- Dianlin Hu et al. Hybrid-domain neural network processing for sparse-view CT reconstruction. *IEEE Transactions on Radiation and Plasma Medical Sciences*, 5(1):88–98, 2021. doi: 10.1109/TRPMS.2020.3011413.
- Kyong Hwan Jin, Michael T. McCann, Emmanuel Froustey, and Michael Unser. Deep convolutional neural network for inverse problems in imaging. *IEEE Transactions on Image Processing*, 26(9):4509–4522, 2017. doi: 10.1109/TIP.2017.2713099.
- James T. Kajiya and Brian P. Von Herzen. Ray tracing volume densities. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 165–174, 1984. doi: 10.1145/800031.808594.
- Avinash C. Kak and Malcolm Slaney. *Principles of Computerized Tomographic Imaging*. Society for Industrial and Applied Mathematics, Philadelphia, 2001.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

- Pavol Klacansky. Open SciVis datasets. <https://klacansky.com/open-scivis-datasets/>, 2017.
- Thomas Kohler. A projection access scheme for iterative reconstruction based on the golden section. In *IEEE Symposium Conference Record Nuclear Science 2004*, volume 6, pages 3961–3965, 2004. doi: 10.1109/NSSMIC.2004.1462462.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. doi: 10.1145/3503250.
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):102:1–102:15, 2022. doi: 10.1145/3528223.3530127.
- Glenn R. Myers, Andrew M. Kingston, Trond K. Varslot, Matthew L. Turner, and Adrian P. Sheppard. Dynamic tomography with a priori information. *Applied Optics*, 50(20):3685–3690, 2011. doi: 10.1364/AO.50.003685.
- Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.
- Mayank Patwari et al. Reducing the risk of hallucinations with interpretable deep learning models for low-dose CT denoising: comparative performance analysis. *Physics in Medicine & Biology*, 68(19):19LT01, 2023. doi: 10.1088/1361-6560/acf5d0.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5301–5310, 2019.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.

- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: 10.1038/323533a0.
- Vincent Sitzmann, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pages 7462–7473, 2020.
- Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.
- Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. doi: 10.1109/TIP.2003.819861.
- Philip J. Withers et al. X-ray computed tomography. *Nature Reviews Methods Primers*, 1:1–17, 2021. doi: 10.1038/s43586-021-00015-4.
- Ruyi Zha, Yanhao Zhang, and Hua Li. NAF: Neural attenuation fields for sparse-view CBCT reconstruction. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 442–452. Springer, 2022.

Appendix A

Supplementary Experimental Details

[TODO: Detailed hyperparameter tables, full benchmark results, extra figures.]

Appendix B

NeCTv2 Hyperparameter Reference

[TODO: Complete listing of all hyperparameters used in each experiment.]